Contents

Dynamic pricing: a two-period symmetric-switching model **2**
Contents . . . 2
1. The economic problem . . . 2
2. Architecture and source of truth . . . 3
3. Utilities: why the history coefficient is signed . . . 3
4. Solving the terminal period . . . 4
5. First-period profit and the effective-cost interpretation . . . 4
6. The numerical root solver and why its solution is unique . . . 5
7. General comparative statics: the implicit-function calculation . . . 6
8. Gamma comparative statics and their interpretation . . . 6
9. Three price ratios that answer different questions . . . 7
10. Using the API and reproducing outputs . . . 8

Dynamic pricing: a two-period symmetric-switching model

This chapter derives the current pricing model and connects it to `model/`, `pricing/`, `comparative_statics/`, numerical evaluation, verification, and exports. `MODEL_CONVENTIONS.md` describes the current conventions. The estimation chapter explains a related but distinct empirical model; this code does not automatically consume its estimates.

Contents

1. Economic setting and state timing
2. File map and source of truth
3. Raw utilities and outside normalization
4. Terminal pricing and value functions
5. First-period pricing by backward induction
6. Numerical solution and uniqueness
7. Comparative statics from implicit differentiation
8. Gamma effects, curvature, and profit
9. Price ratios and conditional interpretations
10. API, figures, tests, and extensions

1. The economic problem

A firm sells one inside product j against an outside alternative 0 over two periods. Each consumer type is fixed and observed for this pricing exercise. The firm has constant marginal cost c and discounts second-period profit by delta. Demand is binary logit. The state records the consumer's last choice, which changes the attractiveness of switching versus staying.

The timing is:

1. Start with observed history $x_0 \in \{j, 0\}$.
2. Set first-period price $p_1(x_0)$; choice x_1 occurs.
3. Observe x_1, then set $p_2(x_1)$.
4. Terminal choice occurs and the horizon ends.

Both second-period branches matter when choosing the first-period price. A consumer who chooses outside in period 1 is still present in period 2. The firm does not receive continuation value only after a sale.

The maintained parameter domain is alpha<0, fixed finite beta/gamma/c, and 0<delta<1. The numerical implementation also permits delta=0 as a static counterfactual. Prices range over all real numbers. A first-period subsidy can be optimal, and the solver deliberately does not impose price at least zero or at least cost. Shares are strictly interior at finite utility indices.

This is a finite-horizon monopoly exercise conditional on a type and initial history. It does not solve rival-firm equilibrium, a household type mixture, an infinite-horizon Bellman equation, or an estimated multi-flavor model.

2. Architecture and source of truth

File/module	Role
`model/symbols.py`	Shared SymPy symbols, shares, branch prices, values, signed histories
`model/assumptions.py`	Human-readable maintained assumptions
`model/utility.py`	State mapping, raw utilities, normalized utilities
`model/shares.py`	Logit expressions and direct share derivatives
`model/profit.py`	Current profit, both continuation branches, total profit
`pricing/period2_foc.py`, `period2_price.py`	Terminal FOC and implicit markup equation
`pricing/period1_foc.py`, `period1_price.py`	Dynamic FOC and continuation adjustment
`comparative_statics/canonical.py`	Production derivative definitions
Other `comparative_statics/*.py`	Named derivative interfaces built from the common model
`model/numerical.py`	Backward induction, root solving, lambdified derivative evaluation
`structural_switching_reference.py`	Independent verification oracle retained from the handoff
`checks/test_symmetric.py`	Current integrated symbolic/numerical verification suite
`make_comparative_static_figures.py`	Benchmark grid, six figure pairs, solution CSV
`make_latex_document.py`	Current mathematical/figure document generation
`export/`	Symbol formatting and LaTeX helpers
`run_all.py`	Current reproducibility entry point

The conceptual dependency order is utilities → shares/profit → FOCs → implicit derivatives → numerical evaluation → figures/document. The numerical evaluator lambdifies expressions from `canonical.py`; the plotting script does not maintain its own hand-copied derivative formulas.

`assumptions.py` stores descriptions; it does not make SymPy infer every inequality. Many shared parameters are declared real, and share symbols positive. Economic domain restrictions must still be supplied when interpreting signs.

Historical notes `DERIVATIVE_CHANGES.md` and `RUN_ALL_DIAGNOSIS.md`, and older figure/CSV files, can concern superseded models. The old entry-only formulation, 0/1 state coefficients, and global ratio-sign claims do not describe current production behavior. The old `z1,z2` numeric interface raises a migration error. Symbolic aliases z1/z2 now mean signed q/q2, not old binary indicators.

3. Utilities: why the history coefficient is signed

Define a state sign

$$q(j) = -1, \qquad q(0) = +1.$$

`signed_state` accepts exactly the strings `"j"` and `"0"`. The latter is a string state name, not a numeric zero passed as a history coefficient.

`raw_utilities(price,q)` returns

$$U_j^{raw} = \beta + \alpha p + \gamma \frac{1+q}{2}, \qquad U_0^{raw} = \gamma \frac{1-q}{2}.$$

Thus a previously outside consumer gets gamma on the inside alternative, while a previously inside consumer gets gamma on the outside alternative. Positive gamma rewards switching in either direction. Negative gamma rewards staying.

Subtracting outside utility from both alternatives preserves choice probabilities:

$$U_j = \beta + \alpha p + q\gamma, \qquad U_0 = 0.$$

This normalization is not the assumption that the outside alternative has no history effect. That effect has been subtracted from the inside utility. Missing it would remove switching incentives after inside consumption.

The inside share is

$$s(p,q) = \frac{1}{1+\exp[-(\beta+\alpha p+q\gamma)]}, \qquad s_p = \alpha s(1-s).$$

At a fixed price, $s_\gamma = qs(1-s)$. This is a direct derivative; after reoptimization the price also changes. `model/shares.py` keeps these partial derivatives separate from total derivatives in `canonical.py`.

4. Solving the terminal period

For a realized second-period history q2, the firm solves

$$V_2(q_2) = \max_{p_2}(p_2 - c)s(p_2, q_2).$$

Differentiate:

$$0 = s_2 + (p_2 - c)\alpha s_2(1 - s_2).$$

Since $s_2 > 0$, divide through to obtain

$$1 + \alpha(1-s_2)(p_2 - c) = 0, \qquad p_2^* = c - \frac{1}{\alpha(1-s_2)}.$$

This is an **implicit** price equation: s2 depends on the unknown optimal price. The code's symbolic `p2_star` is a markup representation, not an explicit solution that can be evaluated with an arbitrary fixed share.

Solve both histories:

$$p_{2j} = p_2^*(j), \quad p_{20} = p_2^*(0), \quad u = s(p_{2j}, -1), \quad v = s(p_{20}, +1),$$

$$V_j = (p_{2j} - c)u, \qquad V_0 = (p_{20} - c)v, \qquad \Delta V = V_j - V_0.$$

Terminal prices exceed cost since alpha<0. At gamma=0 the two branches coincide. For positive gamma the outside-history branch has higher utility intercept, so it has a higher optimized value; $\Delta V < 0$. For negative gamma, inside-history continuation is more valuable and $\Delta V > 0$.

5. First-period profit and the effective-cost interpretation

Let $s = s(p_1, q)$ for the observed initial history. Full expected profit is

$$\Pi(p_1) = s(p_1 - c) + \delta[sV_j + (1-s)V_0].$$

Conditional on each realized branch, Vj and V0 do not depend on p1: second-period price is reset after observing the state, and there are no other carried state variables. They do depend on structural parameters such as gamma. Distinguishing these statements is essential when differentiating profit.

Rewrite the objective as

$$\Pi(p_1) = \delta V_0 + s[p_1 - c + \delta \Delta V].$$

The firm chooses p1 as if it were a static logit monopolist with effective cost $c_{eff} = c - \delta \Delta V$. Its FOC is

$$0 = s + \alpha s(1 - s)[p_1 - c + \delta \Delta V],$$

and the corresponding price equation is

$$p_1^* = c - \frac{1}{\alpha(1 - s)} - \delta \Delta V.$$

When acquisition creates a more valuable continuation state, the firm lowers current effective cost and may subsidize the initial purchase. When a current purchase sends the consumer to a less valuable state, the dynamic adjustment raises price. The firm's gain from inducing a sale is a **difference** in future values, because future profit also exists after outside choice.

The evaluator reports `static_component` as $c - 1/[\alpha(1 - s)]$, using the **dynamic** optimal share. It reports `continuation_adjustment` as $-\delta \Delta V$. Their sum is p1. The separately optimized `static_counterfactual` sets effective cost to c and reoptimizes its share. It generally differs from `static_component`.

At gamma=0, $\Delta V = 0$, so dynamic p1 equals the true static price even with positive delta. Total profit still includes a second-period value.

6. The numerical root solver and why its solution is unique

`optimal_price(intercept, alpha, effective_cost)` solves the same scalar problem for each terminal branch and for period 1. Let A denote the utility intercept and k effective cost. The FOC can be written

$$f(p) = \alpha(p - k) + 1 + \exp(A + \alpha p) = 0.$$

At p=k, f is positive. As p tends to positive infinity, f tends to minus infinity. Moreover,

$$f'(p) = \alpha[1 + \exp(A + \alpha p)] < 0.$$

There is exactly one root above k. The derivative of the profit objective has the sign of f because its remaining factors are positive; profit rises before the root and falls after it. This establishes the global optimum of this scalar unconstrained problem, not merely a stationary point.

The implementation starts a bracket at k, sets initial width `max(2,2/abs(alpha))`, and repeatedly doubles it until the upper endpoint has negative f. `brentq` then finds the root with `xtol=1e-12`. Exponents are clipped to [-745,700] for numerical range, and share computation uses stable `expit`. Clipping is a floating-point safeguard, so extreme calibrations beyond the benchmark warrant additional numerical scrutiny.

`evaluate` validates finite parameters and alpha<0, 0<=delta<1, then:

1. Solves p2j and p20 with effective cost c.
2. Computes u, v, Vj, V0, and their difference.
3. Solves p1 using effective cost c-delta*DeltaV.
4. Computes first-period share and complete two-branch profit.
5. Evaluates lambdified symbolic derivatives at that solution.
6. Reports decomposition terms, counterfactual price, ratio derivatives, and FOC residuals.

There is no need for a simultaneous three-price optimizer: backward induction uses terminal separability. FOC residuals directly check the original profit equations, not only the transformed scalar root.

7. General comparative statics: the implicit-function calculation

`canonical.py` centralizes derivatives for any symbolic parameter theta. Let the direct utility-index derivative at fixed price be

$$D_{2\theta} = \beta_\theta + p_2\alpha_\theta + q_2\gamma_\theta.$$

Differentiating the terminal price equation while allowing its share to respond gives the implemented expression

$$\frac{dp_2^*}{d\theta} = (1 - s_2)c_\theta + \frac{\alpha_\theta}{\alpha^2} - \frac{s_2 D_{2\theta}}{\alpha}.$$

`price2_derivative(theta, share, history, price)` supplies this formula for an arbitrary branch. For beta it is $-s_2/\alpha > 0$; for gamma it is $-q_2 s_2/\alpha$; for delta it is zero. There is no continuation after period 2, so the terminal price cannot depend on discounting.

The envelope theorem eliminates the derivative of the optimized price from the branch-value derivative:

$$\frac{dV_2}{d\theta} = -\frac{s_2 D_{2\theta}}{\alpha} - s_2 c_\theta.$$

This follows by differentiating profit at fixed p2 and using the terminal FOC. `value2_derivative` implements it. It avoids needlessly expanding every price response inside continuation values.

For period 1, with $D_{1\theta} = \beta_\theta + p_1\alpha_\theta + q\gamma_\theta$,

$$\frac{dp_1^*}{d\theta} = (1 - s)c_\theta + \frac{\alpha_\theta}{\alpha^2} - \frac{sD_{1\theta}}{\alpha} - (1 - s)[\delta_\theta \Delta V + \delta \Delta V_\theta].$$

The factors (1-s) reflect the feedback of price into its own demand. Holding the optimal share fixed while differentiating the markup equation would miss that feedback. `price1_derivative` is the production implementation.

For second and mixed derivatives, `total_derivative(expr,theta)` adds explicit parameter differentiation and the chain-rule effects through p1, p2j, p20, s, u, v, Vj, and V0. A plain `sp.diff(expr,gamma)` treats independent share symbols as fixed and therefore is insufficient. `terminal_total_derivative` does the smaller single-branch calculation. Mixed beta-gamma modules use these same helpers rather than a separate model.

8. Gamma comparative statics and their interpretation

8.1 Terminal responses

Substituting signed histories into the preceding formulas gives

$$p'_{2j} = u/\alpha < 0, \qquad p'_{20} = -v/\alpha > 0,$$

$$V'_j = u/\alpha < 0, \qquad V'_0 = -v/\alpha > 0, \qquad \Delta V' = (u + v)/\alpha < 0.$$

Primes in this section mean total derivatives with respect to gamma. More switching preference makes the previously inside branch less attractive and the previously outside branch more attractive. Optimal prices respond in the same directions as those branch demand intercepts.

The reoptimized share derivatives are

$$u' = -u(1 - u)^2, \qquad v' = v(1 - v)^2.$$

They differ from direct share responses -u(1-u) and v(1-v) because prices also move.

8.2 First-period response and conditional signs

The current source simplifies to

$$p_1' = -\frac{qs + \delta(1-s)(u+v)}{\alpha}.$$

For an initially outside consumer q=+1, the numerator is positive, so p1'>0. For an initially inside consumer q=-1, the direct history effect opposes the continuation effect. The price derivative is positive precisely when

$$\delta(1-s)(u+v) > s,$$

zero at equality and negative otherwise. A common global sign across initial histories is therefore unwarranted.

The total first-period share derivative is

$$s' = s(1-s)^2[q - \delta(u+v)].$$

For q=-1 it is negative. For q=+1 its sign depends on the size of the dynamic response. A positive direct demand effect can be offset by optimal repricing.

8.3 Curvature

`d2p1_dgamma2` is a total second derivative. One useful checkable representation is

$$p_1'' = -\frac{1}{\alpha}\left\{s(1-s)^2[q-\delta(u+v)]^2 + \delta(1-s)[-u(1-u)^2 + v(1-v)^2]\right\}.$$

The squared term is nonnegative; the difference of terminal share responses can have either sign. Hence curvature is not established by inspecting only the square. The tests compare this expression both to derivatives of p1' and to second finite differences of fully reoptimized prices.

8.4 Optimized profit

`profit_derivative` applies the first-period envelope and terminal envelopes. It includes both branches and their state probabilities. For gamma:

$$\frac{d\Pi^*}{d\gamma} = \frac{-qs + \delta[su - (1-s)v]}{\alpha}.$$

The absence of explicit p1' does not mean price is fixed in the optimized profit function. The envelope theorem cancels its first-order contribution at the optimum. This formula is not the derivative of only current revenue. Its sign can depend on initial history and calibration; current figures show the conditional curves rather than asserting one universal effect.

9. Three price ratios that answer different questions

The code studies state-matched ratios

$$R_j = p_1(j)/p_{2j}, \qquad R_0 = p_1(0)/p_{20},$$

and an acquisition-conditioned induce/harvest ratio

$$R_{0\to j} = p_1(0)/p_{2j}.$$

The last ratio compares the introductory price for an initially outside consumer to the terminal price conditional on that consumer buying inside in period 1. It is not an unconditional expected second-period price. The matched outside ratio conditions on the outside continuation state instead.

All ratio derivatives use the quotient rule:

$$R' = \frac{p_2 p_1' - p_1 p_2'}{p_2^2}.$$

Increasing both prices does not determine the ratio's direction; their relative changes matter. Negative introductory prices also undermine casual percentage- growth interpretations. If c is sufficiently negative, a terminal price can approach zero and the ratio becomes ill-conditioned or undefined. The benchmark c=0 has positive terminal prices; the generic API does not impose c>=0.

`matched_j`, `matched_0`, and `induce_harvest` are derivative fields, not ratio levels. `evaluate` sets state-inapplicable fields to NaN and supplies convenient `d_matched_ratio` and `d_induce_ratio` fields. Do not interpret NaN there as a solver failure. Ratio levels can be calculated from the price fields when the denominator is safely nonzero.

10. Using the API and reproducing outputs

From `final_draft/supply_code`:

```
from model.numerical import evaluate, solve

r = evaluate(gamma=1.0, initial_state="0", beta=2.0,
             alpha=-1.0, delta=.92, c=0.0)
print(r["p1"], r["p2j"], r["p20"], r["profit"])
print(r["dp1"], r["dprofit"], r["d_matched_ratio"])
prices_and_shares = solve(gamma=1.0, initial_state="j")
```

`evaluate` returns a dictionary. `solve` returns the ordered tuple `(p1,p2j,p20,s,u,v,profit)`. The richer dictionary is safer for analysis because field names preserve the branch meanings.

Dictionary field	Meaning
`gamma`, `initial_state`	Evaluation point and initial history
`p1`, `p2j`, `p20`	Optimal first price and both terminal prices
`s`, `u`, `v`	First-period share and two terminal inside shares
`V2j`, `V20`, `DeltaV`	Optimized branch values and difference
`profit`	Complete discounted two-period profit
`static_component`, `continuation_adjustment`	Algebraic decomposition at the dynamic share
`static_counterfactual`	Independently optimized static price
`dp1`, `d2p1`, `dprofit`	First/second gamma price derivative and profit derivative
`dp2j`, `dp20`	Terminal gamma price derivatives
`d_matched_ratio`, `d_induce_ratio`	Applicable ratio derivatives
`foc1`, `foc2j`, `foc20`	Residuals of original pricing FOCs

The reproducibility command is

```
python3 run_all.py
```

It sequentially runs the symmetric-model unittest suite, makes figures, and generates the LaTeX document with `check=True` subprocess calls. A failed step stops later steps. To compile the supply mathematical document separately:

```
latexmk -cd -pdf -interaction=nonstopmode -halt-on-error \
  writeup_math_graphs/comparative_statics_figures.tex
```

These commands regenerate research artifacts. The tutorial PDFs described in the overview are separate documentation artifacts.

Benchmark and figures

The benchmark is beta=2, alpha=-1, delta=.92, c=0. A grid of 601 gamma values from -3 to 3 is solved for each of two initial histories, giving 1,202 rows in `writeup_math_graphs/figures/solutions.csv`. Each figure is exported as PNG and PDF:

Stem	Plotted quantity
`matched_price_ratio_derivative`	Both state-matched ratio derivatives
`induce_harvest_price_ratio_derivative`	Acquisition-conditioned ratio derivative
`profit_by_gamma`	Optimized profit by initial history
`gamma_profit_by_gamma`	Optimized profit derivative
`optimal_prices_by_gamma`	First prices by initial state and both terminal prices
`gamma_p1_by_gamma`	First-price gamma derivatives

The terminal prices do not depend on x0 once x1 is realized, so the price-level figure can take terminal curves from either initial-state collection. Grid generation asserts maximum absolute FOC residual at most `1e-9`. Old files with other stems can remain in the directory and are not evidence that those old plots belong to the current model.

What verification establishes

`checks/test_symmetric.py` checks raw utility normalization, FOCs differentiated from literal profit, branch-probability derivatives, envelope identities, implicit-function identities, and gamma curvature. It compares the numerical implementation to the independent reference across the full benchmark grid.

At several other calibrations and gamma values, it compares analytic derivatives to centered finite differences of reoptimized solutions. An independent scalar profit optimizer checks p1 against direct maximization of complete two-branch profit. It also checks alpha/beta/delta/c derivatives, a mixed beta-gamma derivative, static limits, terminal-state symmetry, and negative introductory prices under strong habit.

These layers address different errors. Algebraic identities catch missing terms; finite differences catch incorrect total differentiation or argument mapping; the independent optimizer catches a solver implementing the wrong objective; static/symmetry tests catch state-convention regressions. Agreement with a reference alone would not establish the economic validity of assumptions.

For checks without regenerating figures:

```
python3 -m unittest -v checks.test_symmetric
```

Extending the model

If a parameter changes utility, update raw utilities and normalized shares, derive the profit FOCs, then propagate the change through canonical derivatives and numerical argument mapping. Generic derivative helpers assume the present linear index and transition structure; they are not automatically correct for a new state transition, nonlinear price term, or endogenous cost.

Adding a price floor requires a constrained solution and piecewise comparative statics: the current interior formulas cannot simply be evaluated at a clipped price. Adding persistent unobserved types requires a policy specifying whether the firm observes types or updates beliefs, because type-specific optimal prices cannot generally be averaged to obtain a pooled optimum. Extending to many periods requires a richer continuation recursion.

Finally, using estimated yogurt parameters requires an explicit mapping of alternatives, price units, normalizations, history units, and information. The current benchmark is a calibrated theoretical exercise with a binary last-choice state; it is not a direct supply-side estimate from the three-flavor demand output.